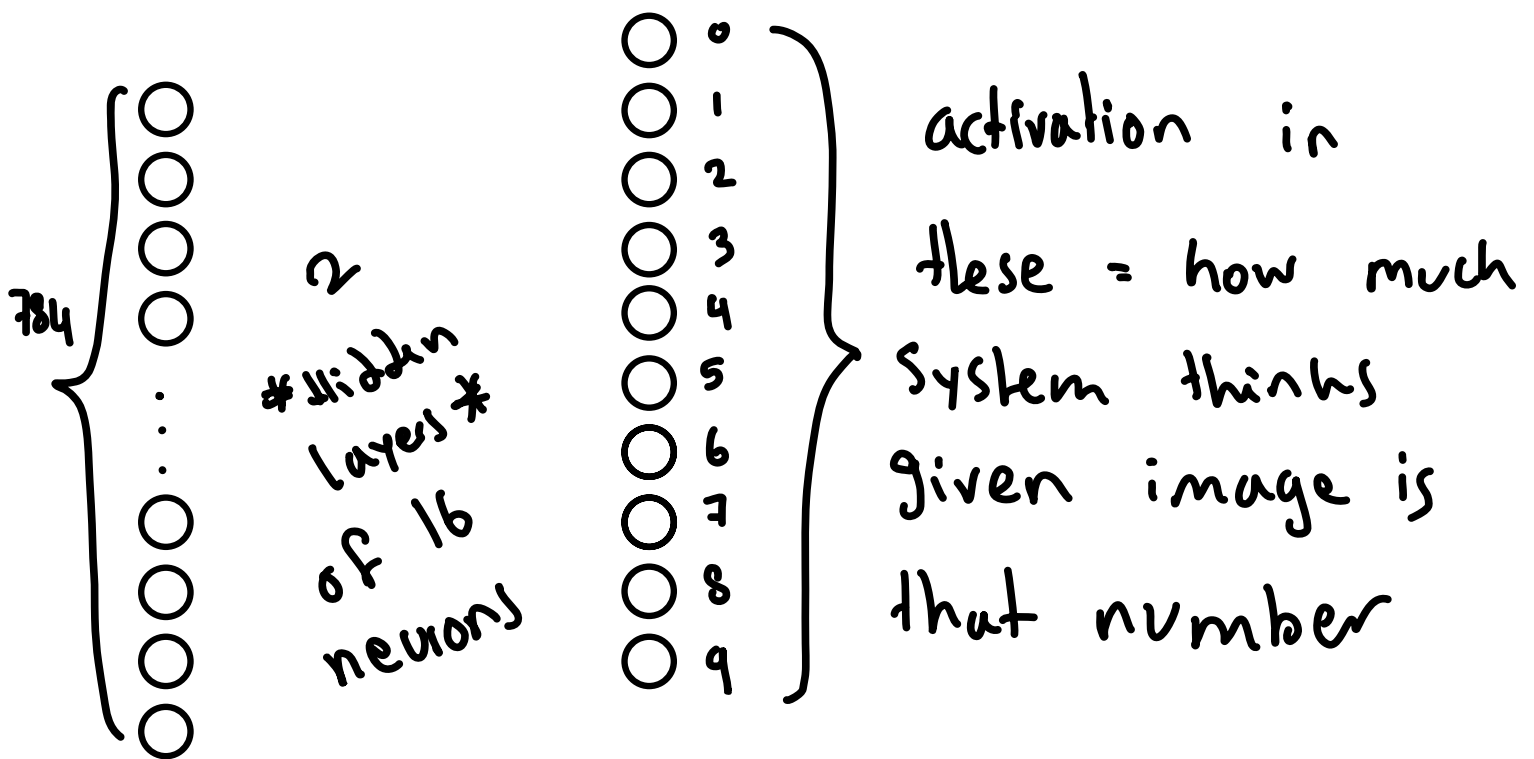


what is a neural network?

- neural network to recognize handwritten digits : MNIST Problem
- Plain vanilla : multilayer perceptron
- neuron $\rightarrow$ thing holds a number $[0,1]$
- ex: network starts w/ bunch of neurons corresponding to 28×28 pixels
- 784 neurons total, holding grayscale ^{value of} pixel
- ~~x~~ in neuron = activation

- neuron lit up when high activation
- these 784 are 1st layer of network



- a lot of room to experiment w/ structure
- activations of 1 layer, determine activations of next layer
- heart: how activations in one layer affect the next layer

- network in video already trained i
 - if u feel in image lighting
all 784 neurons of input layer...
 - that causes patter in next layers
Until system can predict
 - brightest neuron in output
layer is the network's choice

• why layers?

→ sub problems.

- goal is to have some mechanism to combine patterns
- ex: hope one neuron finds straight edge up top.

→ what parameters should the network have?

→ dials to tweak to capture pattern

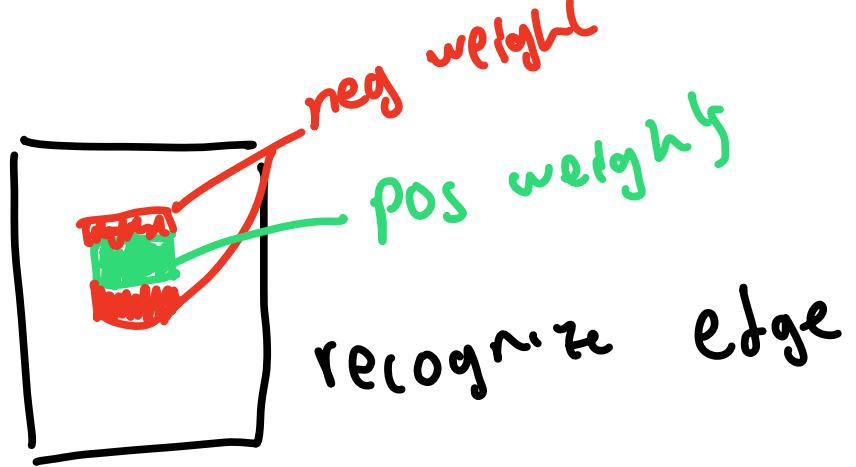
→ assign a weight to each connection b/w our neuron & the next layer



weighted sum:

$$w_1 a_1 + w_2 a_2 + \dots + w_n a_n$$

→ Positive weights
for this region,
zero elsewhere

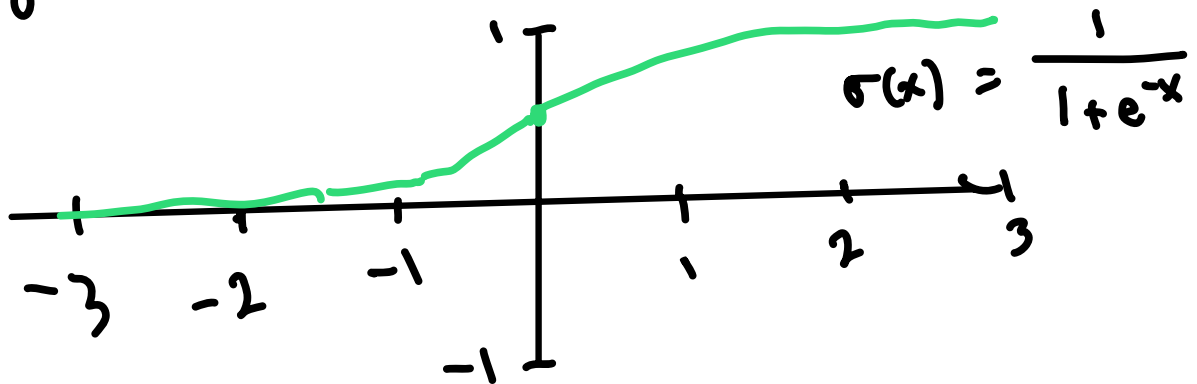


→ sum largest when middle pixel
bright & edge pixels dark.
= "recognize edge"

→ weighted sum > 1 . so func()
brings back to $[0, 1]$

→ Sigmoid function is very common:

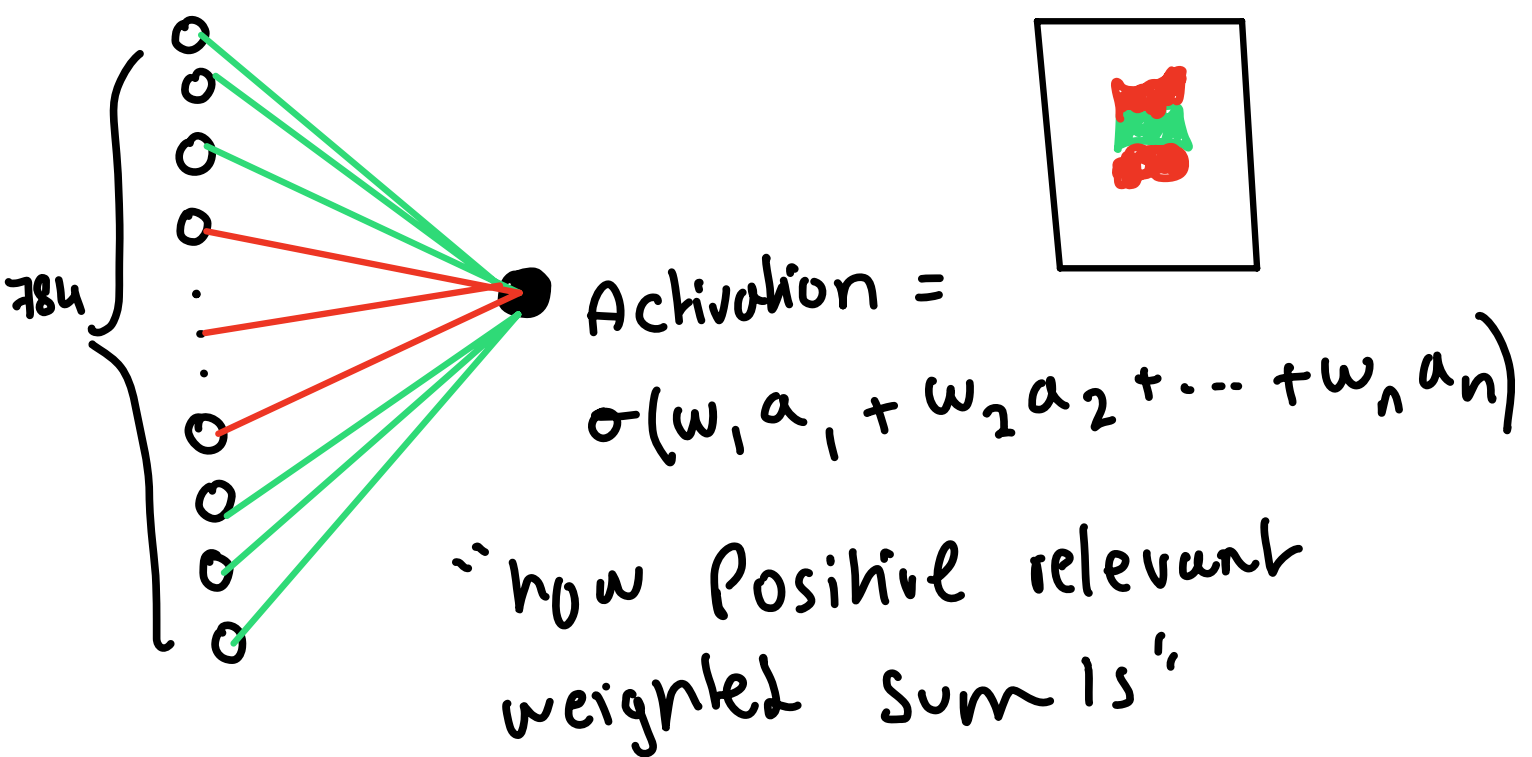
logistic
curve



→ very neg close to zero

→ very pos close to 1

→ activation of our neuron



- but maybe neuron should only light up when weighted sum > 10 , not 1...
- you want a bias for it to be inactive

- add your **bias** by sigmoid squish func
 $\sigma(w_1 a_1 + w_2 a_2 + \dots + w_n a_n - 10)$

- weights = what pattern in pixels
neurons are picking up on

- biases = how high weighted sum
needs to be before it lights up
meaningfully

- every neuron 1st layer connected ^{to} next

- each connection has weight & bias

- $784 \rightarrow 16$ is 784×16 weights & 16 biases
neurons neurons

- that's just the 1st layer ✓

- total $784 \rightarrow 16 \rightarrow 16 \rightarrow 10$ has 13k weights & biases, knobs to turn for network to behave differently
- learning = finding right weights & biases
- imagine setting these by hand---
- to make it easier to read,

All activations of one layer as a vector. then all weights as a matrix,

$$\begin{bmatrix} w_{0,0} & w_{0,1} & \dots \\ \vdots & \vdots & \vdots \\ w_{n,0} & w_{n,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix}$$

row = connections
b/w 1 layer &
1 neuron in next
layer

weighted sum = matrix vector product

$$\begin{bmatrix} \end{bmatrix} \begin{bmatrix} \end{bmatrix} = \begin{bmatrix} \end{bmatrix}$$

next layer
activations

• machine learning from lin alg

• biases in own vector, wrap in sigmoid

$$\sigma \left(\begin{bmatrix} \text{weights} \end{bmatrix} \begin{bmatrix} \text{activations} \end{bmatrix} + \begin{bmatrix} \text{biases} \end{bmatrix} \right)$$

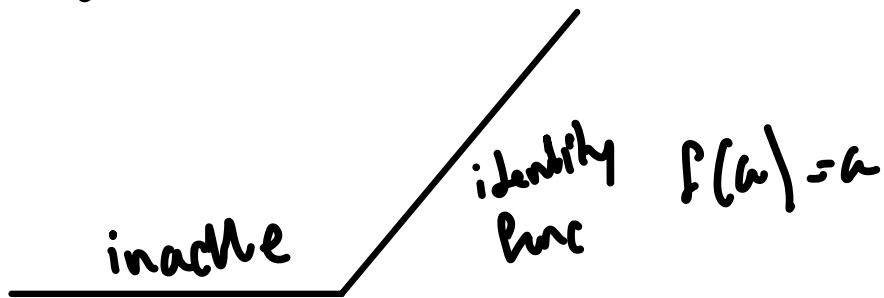
$$a^{(1)} = \sigma(W a^{(0)} + b) \quad // \text{ activations of layer 1 given layer 0}$$

- libraries optimize matrix multiplication
- each neuron is a function,
takes in output of all neurons prev
spit out $[0, 1]$
- whole network takes in 784 nums, ^{output} 10.
- network has 13k weights & biases
to pick up on patterns, &
iterate on matrix-vector products
& sigmoid squish-like functions
- How to train? learn weights
& biases?

- sigmoid replaced by ReLU

$$\text{ReLU}(a) = \max(0, a)$$

- biological analogy



- sigmoid hard to train, relu better.